

Praktikum 4: Binäre Suchbäume (BST)

Binäre Suchbäume erlauben das effiziente Suchen, Einfügen und Löschen von Schlüsseln in $O(\log n)$ - vorausgesetzt, der Baum ist ausgewogen. Ob das tatsächlich der Fall ist, ist keineswegs garantiert. In diesem Praktikum untersuchen Sie dieses Verhalten experimentell, lernen ein Visualisierungswerkzeug kennen und erweitern die vorhandene Implementierung um das Löschen im AVL-Baum.

Die Klassen `BinaryTree`, `BinaryTreeNode`, `AVLTree` und `AVLTreeNode` aus der Vorlesung stehen Ihnen vollständig zur Verfügung.

Visualisierung mit Graphviz

Graphviz (<https://graphviz.org/>) ist ein quelloffenes Werkzeug zur automatischen Berechnung von Graphen- und Baumlayouts. Aus einer textuellen Beschreibung (z.B. `graph.gv`) in der DOT-Sprache erzeugt es unter anderem PDF-Dateien:

```
dot -Tpdf graph.gv > graph.pdf
```

Installieren Sie Graphviz auf Ihrem Rechner.

Laden Sie anschließend die Folge `seq0.txt` in einen `BinaryTree` und rufen Sie `graph_traversal()` auf. Öffnen Sie das erzeugte PDF und beantworten Sie:

- Welcher Knoten ist die Wurzel?
- Wie viele Blattknoten hat der Baum?
- Wie groß ist die Höhe des Baums?

Traversierungen

Führen Sie auf demselben Baum die folgenden Traversierungen aus und vergleichen Sie die Ausgaben:

- `in_order_traversal`
- `level_order_traversal`

Beantworten Sie:

- Welche Traversierung gibt die Schlüssel in aufsteigend sortierter Reihenfolge aus, und warum ist das bei einem BST zwangsläufig so?
- Was zeigt `level_order_traversal`, das `in_order_traversal` nicht erkennen lässt?
- Schauen Sie sich den Quellcode von `graph_traversal` an. Welche Traversierung wird intern genutzt, und wozu dienen die Parameter `level` und `line`, die an den Callback übergeben werden?

Entartung des BST

Fügen Sie die Folge [1, 2, 3, 4, 5, 6, 7, 8] in einen neuen **BinaryTree** ein und visualisieren Sie das Ergebnis mit Graphviz.

Vergleichen Sie den Baum mit dem aus Aufgabe 1 und beantworten Sie:

- Wie viele Vergleiche benötigt **search(8)** in diesem Baum?
- Wie viele Vergleiche wären es bei einem vollständig ausgewogenen Baum mit 8 Knoten?
- Welche Eingabereihenfolge würde beim BST stets einen ausgewogenen Baum erzeugen?
- Was müsste eine Datenstruktur (wie z.B. ein AVL-Baum) garantieren, um dieses Problem grundsätzlich zu lösen?

Löschen im AVL-Baum

Bauen Sie einen AVL-Baum mit den Werten [10, 5, 15, 3, 7, 12, 20, 1, 4, 6, 8] auf und löschen Sie jeweils einen Knoten für alle drei Fälle:

- Löschen eines Blattknotens
- Löschen eines Knotens mit genau einem Kind
- Löschen eines Knotens mit zwei Kindern

Visualisieren Sie den Baum jeweils vor und nach dem Löschen mit Graphviz und überprüfen Sie, dass der AVL-Baum nach jeder Operation korrekt ausgewogen ist. Beantworten Sie anhand des Quellcodes von **delete** in **AVLTree**:

- Nach dem Löschen wird **node.parent = parent** neu gesetzt. Warum ist das notwendig? Was würde ohne diese Zeile passieren?
- Die Rebalancierung beginnt mit **balance(parent)**, nicht mit **balance(node)**. Warum?

Komplexitätsanalyse

Leiten Sie die Laufzeit-Komplexität von **search** für die folgenden drei Fälle theoretisch her:

- AVL-Baum: Formulieren Sie die Rekurrenz für die Suche und lösen Sie sie mit dem Master-Theorem.
- BST (Worst Case - sortierte Eingabe): Formulieren Sie die Rekurrenz und lösen Sie sie.
- BST (Zufällige Eingabe): Geben Sie die erwartete Höhe an und begründen Sie sie intuitiv.

Messen Sie anschließend die Anzahl der Vergleiche bei **search** für $n \in \{10, 100, 500\}$ und tragen Sie die Werte in eine Tabelle ein:

n	BST zufällig	BST sortiert	AVL sortiert
10			
100			
500			

Stimmen die gemessenen Werte mit den theoretisch hergeleiteten Komplexitätsklassen überein?